



Institución Universitaria

INGENIERÍA DE SISTEMAS

ASEGURAMIENTO DE LA CALIDAD

ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE

PRUEBAS CASO FINAL SOBRE CNARIOS

FELIPE LONDOÑO GARCÍA

JUAN PABLO GÓMEZ RAMÍREZ

ZIUVAR RUIZ ÁLVAREZ

GRUPO 190304017-1

INSTITUTO TECNOLÓGICO METROPOLITANO

MEDELLÍN

2026

Tabla de contenido

- 1. Introducción
- 2. Contexto del proyecto
- 3. Requerimientos y criterios de calidad
- 4. Plan de pruebas
- 5. Diseño de casos de prueba
- 6. Ejecución y evidencias de pruebas
- 7. Automatización de pruebas
- 8. Gestión de defectos
- 9. Pruebas de seguridad
- 10. Métricas de calidad
- 11. Reflexión y lecciones aprendidas
- 12. Conclusiones
- 13. Anexos

1. Introducción

El presente informe documenta la planificación, diseño, ejecución y análisis de actividades de aseguramiento de calidad aplicadas sobre el sistema web Cnarios. El objetivo es evidenciar el uso integrado de pruebas funcionales y no funcionales, automatización, gestión de defectos, seguridad, métricas de calidad y análisis de resultados sobre un sistema funcional real.

Se seleccionó Cnarios porque ofrece escenarios prácticos para automatización web, incluyendo iframes, múltiples ventanas, enlaces, tablas, paginación, filtros y contenido educativo. Esto permite aplicar técnicas de prueba de caja negra y automatización con Playwright sobre funcionalidades observables, sin depender de acceso al código fuente del producto.

El trabajo fue desarrollado sobre un repositorio de pruebas propio, disponible en GitHub, donde se organizan scripts automatizados, evidencias, reportes, configuraciones, métricas y documentación técnica.

Elemento	Descripción
Sistema evaluado	Cnarios
URL del sistema	https://www.cnarios.com
Repositorio	https://github.com/ziuvar/PROYECTO-FINAL-ASEGURAMIENTO.git
Tipo de evaluación	Caja negra sobre sitio público
Herramientas principales	Playwright, TypeScript, Docker, k6, OWASP ZAP, SonarQube

2. Contexto del proyecto

2.1 Funcionalidad general del sistema

Cnarios es una plataforma web orientada al aprendizaje y práctica de pruebas automatizadas. Su funcionalidad principal consiste en ofrecer conceptos, desafíos y blogs relacionados con automatización de pruebas. El sitio permite practicar flujos comunes en aplicaciones web modernas, como formularios dentro de iframes, apertura de ventanas, validación de enlaces, tablas de datos, paginación y filtros.

2.2 Problema que resuelve

El sistema facilita que estudiantes y testers practiquen automatización sobre escenarios cercanos a situaciones reales. Esto reduce la brecha entre ejercicios teóricos y pruebas aplicadas en aplicaciones web con interacción dinámica.

2.3 Arquitectura observada

El código fuente de Cnarios no se encuentra disponible para el equipo, por lo cual el análisis se realiza como caja negra. A partir de la observación, el sistema se comporta como una aplicación web cliente-servidor accesible mediante HTTPS.

Usuario / Navegador

|

| HTTPS

v

Aplicación Web Cnarios

|

| Recursos estáticos, rutas y contenido

v

Servicios o backend no visible

|

v

Almacenamiento o CMS no accesible

Capa	Descripción observable
Frontend	Aplicación web pública cargada en navegador.
Backend	No observable directamente. Se infiere un servicio que entrega rutas, recursos y contenido.
Base de datos	No accesible. Se infiere almacenamiento de conceptos, desafíos, blogs y preguntas.
Infraestructura	Sitio público disponible mediante HTTPS.
Repositorio QA	Proyecto de automatización y documentación desarrollado por el equipo.

2.4 Alcance funcional cubierto

- Conceptos: iFrames, Multi Window, Links y Table.
- Desafíos: E-commerce Product Listing & Pagination y E-commerce Product Filtering & Search.
- Blogs: carga de artículos principales.
- Navegación general: rutas principales y enlaces de secciones.
- No funcionales: rendimiento, estabilidad, usabilidad básica, seguridad pasiva y compatibilidad.

Fuera de alcance: pruebas unitarias internas, integración real de backend/base de datos y corrección directa de defectos del producto, debido a que no se dispone del código fuente de Cnarios.

3. Requerimientos y criterios de calidad

3.1 Requisitos funcionales

ID	Requisito	Criterio de aceptación
RF01	Navegar entre conceptos.	Cada concepto carga título, descripción y pestaña interactiva.
RF02	Interactuar con iframes, enlaces, ventanas y tablas.	Los elementos responden sin errores críticos.
RF03	Usar desafíos de paginación y filtros.	La paginación cambia productos y los filtros devuelven resultados coherentes.
RF04	Consultar artículos del blog.	Los artículos cargan correctamente y son accesibles.
RF05	Mantener navegación principal funcional.	Home, Concepts, Challenges y Blogs redirigen sin errores HTTP 5xx.

3.2 Requisitos no funcionales

ID	Requisito	Criterio de aceptación
RNF01	Rendimiento.	Las páginas críticas deben cargar en menos de 3 segundos.
RNF02	Usabilidad.	La interfaz debe ser comprensible para usuarios nuevos.
RNF03	Fiabilidad.	No deben presentarse errores críticos de consola ni HTTP 5xx.
RNF04	Seguridad.	No deben existir vulnerabilidades críticas evidentes en revisión pasiva.
RNF05	Compatibilidad.	Los flujos principales deben ejecutarse en Chrome y Firefox.

3.3 Atributos de calidad analizados

Atributo	Análisis aplicado
Funcionalidad	Validación de conceptos, desafíos, blogs y navegación principal.
Usabilidad	Revisión de claridad visual, mensajes y facilidad de uso.
Fiabilidad	Monitoreo de consola y respuestas HTTP durante navegación.
Rendimiento	Medición con Playwright, DevTools y k6.
Seguridad	Evaluación OWASP Top 10 con ZAP y revisión pasiva.
Mantenibilidad	Uso de POM, separación de scripts y análisis SonarQube sobre repositorio QA.

4. Plan de pruebas

El plan de pruebas se estructuró siguiendo lineamientos IEEE/ISO, considerando objetivos, alcance, niveles de prueba, estrategias, técnicas, recursos, ambientes, criterios de entrada y salida, y cronograma de ejecución.

4.1 Objetivos de prueba

- Verificar que Cnarios cumple los requisitos funcionales y no funcionales definidos.
- Detectar defectos relacionados con navegación, formularios, iframes, ventanas, enlaces, tablas, paginación y filtros.
- Evaluar tiempos de carga, estabilidad básica y postura de seguridad observable.
- Implementar automatización reproducible con Playwright y TypeScript.
- Documentar evidencias, defectos, métricas y recomendaciones.

4.2 Tipos, niveles y estrategias

Tipo/Nivel	Aplicación en el proyecto
Unitarias	No aplican al producto por caja negra. Aplican al repositorio QA si se amplían utilidades.
Integración	Se valida integración observable entre SPA, rutas, recursos y navegador.
Sistema	Principal nivel de prueba mediante flujos completos desde la interfaz web.
Aceptación	Validación de criterios RF/RNF contra casos manuales y automatizados.
Funcionales	Conceptos, desafíos, blogs y navegación.
No funcionales	Rendimiento, fiabilidad, usabilidad, compatibilidad y seguridad.
Automatizadas	Playwright con Page Object Model.
Exploratorias	Navegación libre, consola, enlaces y observación de comportamiento.

4.3 Técnicas aplicadas

Técnica	Uso aplicado
Partición de equivalencia	Datos válidos e inválidos en formularios y filtros por categoría.
Valores límite	Umbral de 3 segundos, tasa de fallos HTTP menor al 1 %, límites de navegación/paginación.
Tabla de decisión	Comportamiento esperado al combinar categoría, búsqueda y reset de filtros.
Caja negra	Evaluación del comportamiento observable sin acceso al código fuente del sistema.
Exploratoria	Identificación de defectos no previstos durante navegación manual.

4.4 Recursos y ambiente

Recurso	Uso
Playwright + TypeScript	Automatización funcional y no funcional básica.
Docker Desktop	Ejecución de k6, OWASP ZAP y SonarQube/SonarScanner.
k6	Prueba básica de carga y tiempo de respuesta.
OWASP ZAP	Escaneo pasivo de seguridad basado en OWASP Top 10.
SonarQube	Análisis estático del repositorio de pruebas.
GitHub Actions	Propuesta de integración continua.
Equipo humano	QA Lead, tester manual, automatizador y analista de seguridad/rendimiento.

4.5 Cronograma

Actividad	Responsable	Inicio	Fin
Revisión de requisitos y plan	QA Lead	14/may/2026	15/may/2026
Diseño de casos manuales	Tester manual	15/may/2026	17/may/2026
Automatización	Tester automatizador	17/may/2026	20/may/2026
Ejecución manual	Tester manual	18/may/2026	21/may/2026

Actividad	Responsable	Inicio	Fin
Ejecución automatizada	Tester automatizador	20/may/2026	22/may/2026
Rendimiento y seguridad	Analista QA	21/may/2026	22/may/2026
Análisis y métricas	Equipo QA	21/may/2026	23/may/2026
Informe final	QA Lead y equipo	23/may/2026	24/may/2026

5. Diseño de casos de prueba

Se diseñaron 10 casos manuales y 5 casos automatizados funcionales, además de pruebas no funcionales asociadas a rendimiento, consola y respuestas HTTP. Cada caso se relaciona con requisitos funcionales o no funcionales.

5.1 Casos manuales

ID	Nombre	Req.	Estado
TCM-01	Navegación a conceptos	RF01, RF05	Aprobado
TCM-02	Interacción con iframe	RF02	Aprobado con observación
TCM-03	Apertura de nueva ventana	RF02	Aprobado
TCM-04	Verificación de enlaces	RF02	Aprobado
TCM-05	Funcionalidad de tabla	RF02	Aprobado
TCM-06	Navegación del blog	RF04	Aprobado
TCM-07	Paginación	RF03	Aprobado
TCM-08	Filtro por categoría	RF03	Aprobado
TCM-09	Tiempos de carga	RNF01	Aprobado
TCM-10	Errores JavaScript/HTTP	RNF03	Aprobado

5.2 Casos automatizados

ID	Nombre	Req.	Archivo
TCA-01	Navegación y conceptos	RF01	tests/cnarios.automated.spec.ts
TCA-02	Interacción con iframe	RF02	tests/cnarios.automated.spec.ts
TCA-03	Multi-window	RF02	tests/cnarios.automated.spec.ts
TCA-04	Paginación	RF03	tests/cnarios.automated.spec.ts
TCA-05	Filtrado	RF03	tests/cnarios.automated.spec.ts
RNF01	Tiempos de carga	RNF01	tests/ cnarios.nonfunctional.spec.ts
RNF03	Consola y HTTP 5xx	RNF03	tests/ cnarios.nonfunctional.spec.ts

5.3 Matriz de trazabilidad

Req.	Descripción	Casos/Evidencias
RF01	Navegar entre conceptos	TCM-01, TCA-01
RF02	Interactuar con iframe, enlaces, ventanas y tablas	TCM-02 a TCM-05, TCA-02, TCA-03
RF03	Usar desafíos de paginación y filtros	TCM-07, TCM-08, TCA-04, TCA-05
RF04	Consultar blogs	TCM-06
RF05	Navegación principal	TCM-01, RNF03
RNF01	Rendimiento	TCM-09, k6, Playwright
RNF03	Fiabilidad	TCM-10, RNF03 automatizada
RNF04	Seguridad	OWASP ZAP, revisión pasiva, SonarQube

6. Ejecución y evidencias de pruebas

Se ejecutó la suite automatizada con Playwright mediante el comando `npm run test:compat`. La ejecución cubrió navegadores Chrome y Firefox, validando navegación, iframe, apertura de ventanas, paginación, filtros, tiempos de carga y errores HTTP/JavaScript. Como evidencia se adjuntan capturas del reporte HTML de Playwright, la consola de ejecución y los artefactos generados por la herramienta.

```
npm run test:compat
```

```
PS C:\Users\ziuvar\Desktop\ASEGURAMIENTO FINAL> npm.cmd run test:compat

> aseguramiento-final@1.0.0 test:compat
> playwright test --project=chrome --project=firefox

Running 14 tests using 4 workers

  ✓ 1 ...CA-01 navega por conceptos y valida titulos visibles (4.4s)
  ✓ 2 ...01 paginas criticas cargan bajo umbral de 3 segundos (8.2s)
  ✓ 3 ...01 paginas criticas cargan bajo umbral de 3 segundos (9.3s)
  ✓ 4 ...CA-01 navega por conceptos y valida titulos visibles (4.2s)
  ✓ 5 ...CA-02 interactua con el formulario dentro del iframe (2.9s)
  ✓ 6 ...CA-02 interactua con el formulario dentro del iframe (2.8s)
  ✓ 7 ...ulti Window y conserva la pagina principal operativa (3.2s)
  ✓ 8 ...ulti Window y conserva la pagina principal operativa (3.2s)
  ✓ 9 ...pal no muestra errores HTTP 5xx ni errores JS graves (7.1s)
  ✓ 10 ...pal no muestra errores HTTP 5xx ni errores JS graves (7.7s)
  ✓ 11 ...valida avance y retroceso de paginacion en productos (2.8s)
  ✓ 12 ...valida avance y retroceso de paginacion en productos (2.9s)
  ✓ 13 ...5 filtra productos por categoria y restaura la lista (3.9s)
  ✓ 14 ...5 filtra productos por categoria y restaura la lista (3.9s)

14 passed (18.8s)

To open last HTML report run:

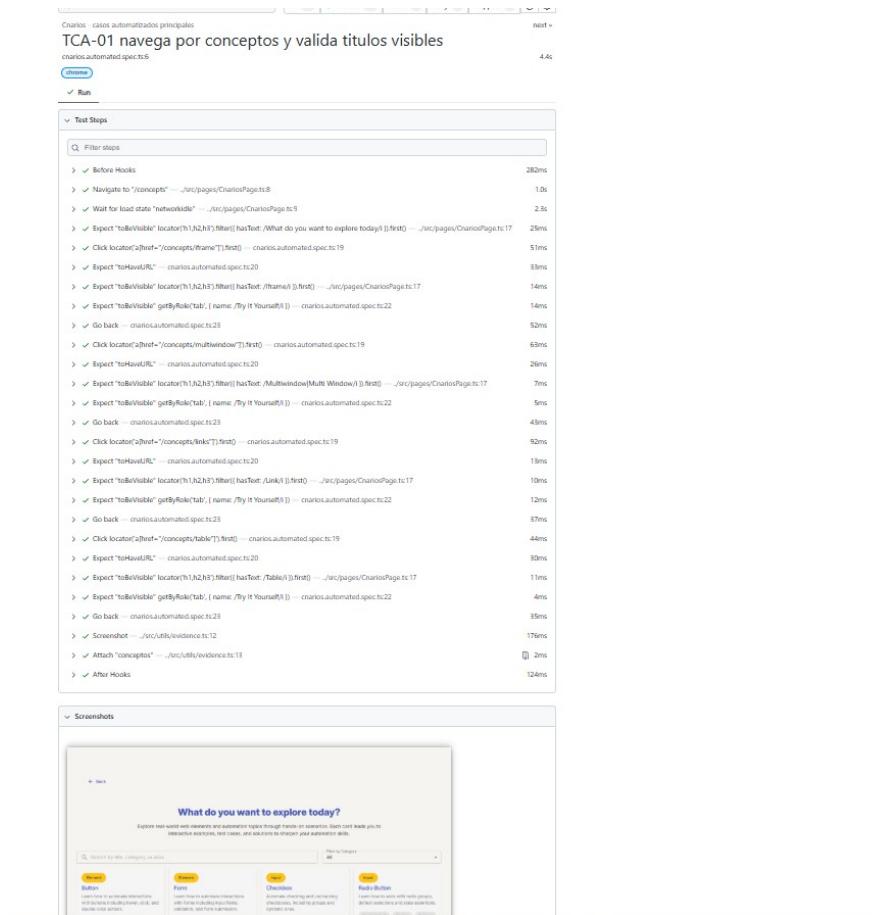
npx playwright show-report evidencias\playwright-report

PS C:\Users\ziuvar\Desktop\ASEGURAMIENTO FINAL> 
```

Evidencia EV-AUTO-01: ejecución de pruebas Playwright desde consola.

Test Name	Duration	Status
casos automatizados principales - TCA-01 navega por conceptos y valida titulos visibles	4.4s	Passed
casos automatizados principales - TCA-02 interactua con el formulario dentro del iframe	2.9s	Passed
casos automatizados principales - TCA-03 abre nueva pestaña desde Multi Window y conserva la pagina principal operativa	3.2s	Passed
casos automatizados principales - TCA-04 valida avance y retroceso de paginacion en productos	2.8s	Passed
casos automatizados principales - TCA-05 filtra productos por categoria y restaura la lista	3.9s	Passed
casos automatizados principales - TCA-01 navega por conceptos y valida titulos visibles	4.2s	Passed
casos automatizados principales - TCA-02 interactua con el formulario dentro del iframe	2.8s	Passed
casos automatizados principales - TCA-03 abre nueva pestaña desde Multi Window y conserva la pagina principal operativa	3.2s	Passed
casos automatizados principales - TCA-04 valida avance y retroceso de paginacion en productos	2.8s	Passed
casos automatizados principales - TCA-05 filtra productos por categoria y restaura la lista	3.9s	Passed
pruebas no funcionales basicas - RNFO1 paginas criticas cargan bajo umbral de 3 segundos	8.2s	Passed
pruebas no funcionales basicas - RNFO3 navegacion principal no muestra errores HTTP 5xx ni errores JS graves	7.1s	Passed
pruebas no funcionales basicas - RNFO1 paginas criticas cargan bajo umbral de 3 segundos	9.3s	Passed
pruebas no funcionales basicas - RNFO3 navegacion principal no muestra errores HTTP 5xx ni errores JS graves	7.7s	Passed

Evidencia EV-AUTO-02: reporte de ejecución automatizada en Playwright.

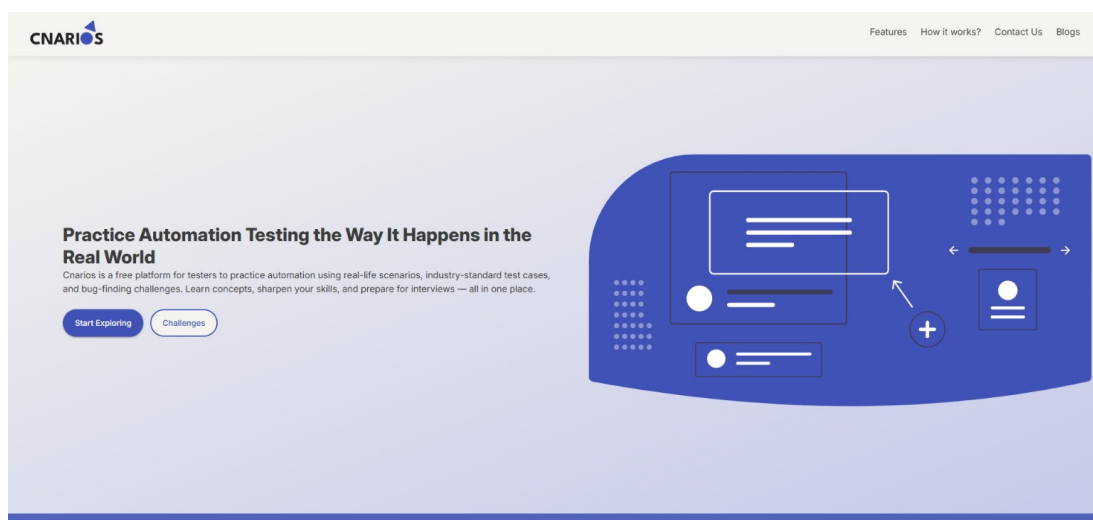


Evidencia EV-AUTO-03: detalle de artefactos y trazas de Playwright.

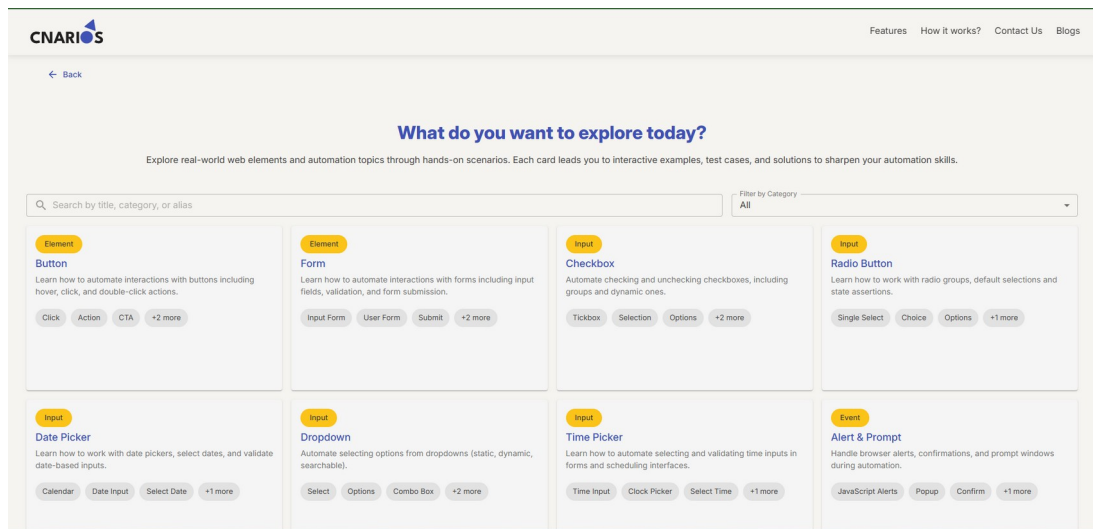
6.1 Evidencias de casos manuales

TCM-01: Navegación a conceptos

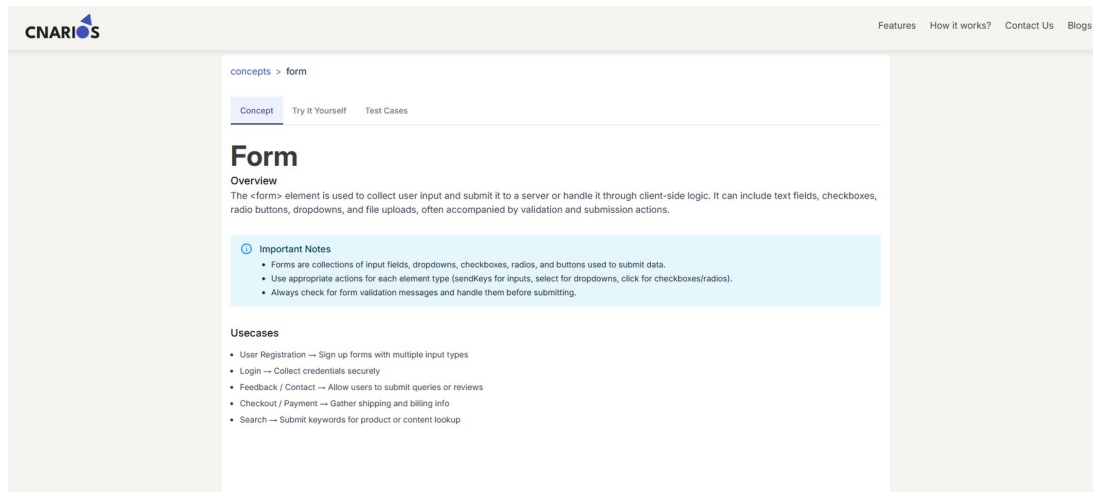
Se validó el acceso a la página principal y a la sección de conceptos.



EV-MAN-01: página principal de Cnarios.



EV-MAN-02: listado de conceptos disponible.



EV-MAN-03: concepto Form visible desde el portal.

concepts > iframe

Concept

Try It Yourself

Test Cases

Iframe

Overview

The <iframe> element embeds another HTML page within the current document.

Important Notes

- iFrames are HTML documents embedded within another page and need to be switched into before interacting.
- Use `switchTo().frame()` or equivalent to access elements inside the iFrame.
- Always switch back to the main document after interacting using `switchTo().defaultContent()`.
- Check if the iFrame is visible and fully loaded before performing actions.

Usecases

- Embedding external content
- Sandboxed environments for forms or ads

EV-MAN-04: concepto Iframe cargado correctamente.

TCM-02: Interacción con iframe

Se diligenció el formulario embebido dentro del iframe con datos de prueba.

concepts > iframe

Concept

Try It Yourself

Test Cases

HTML Editor Cnario

intermediate

Document Structure

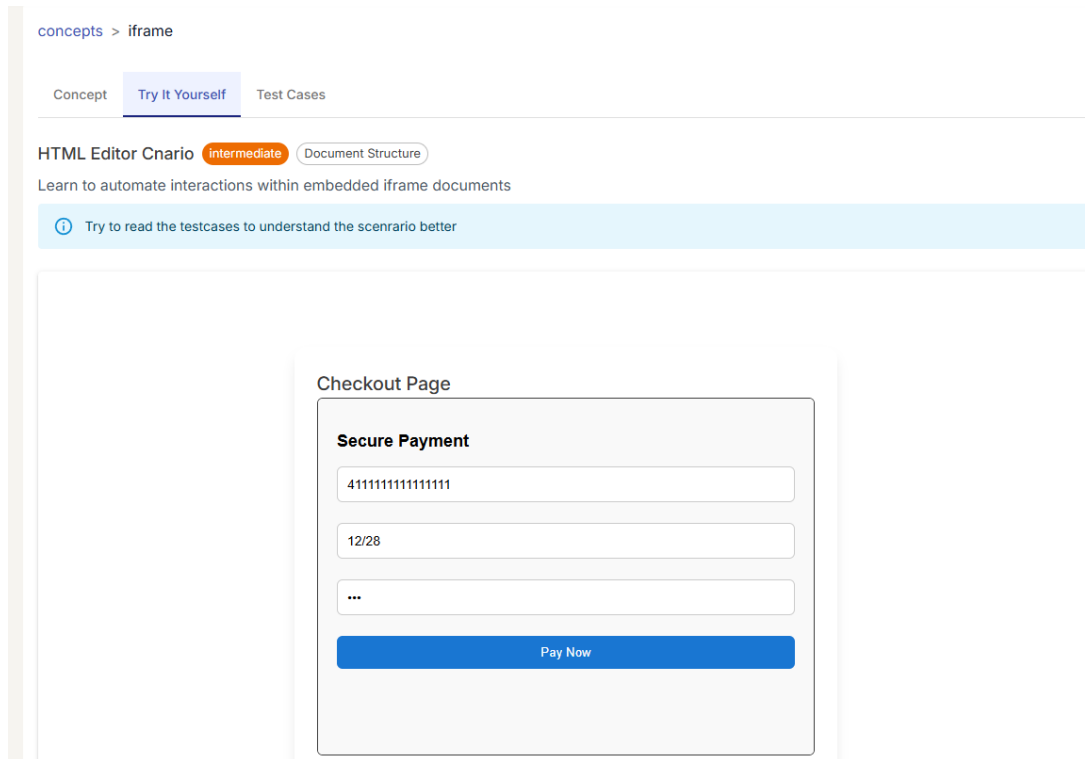
Learn to automate interactions within embedded iframe documents

Try to read the testcases to understand the scenario better

Checkout Page

Secure Payment

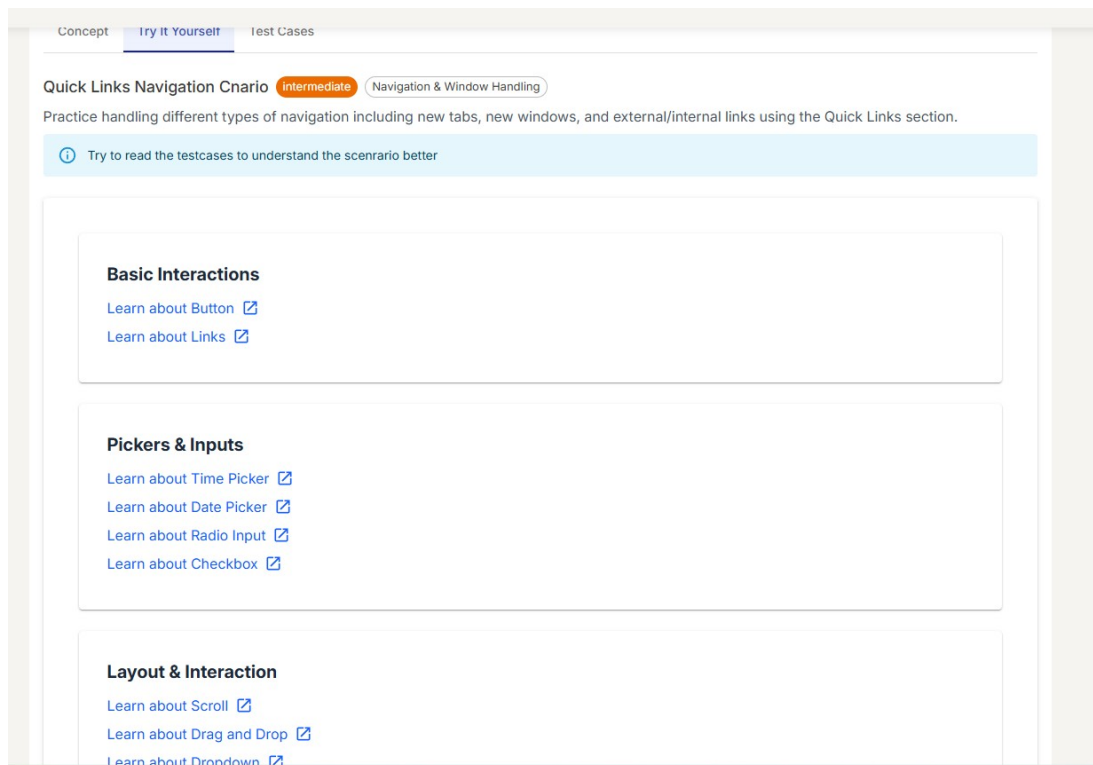
EV-MAN-05: formulario de pago dentro del iframe.



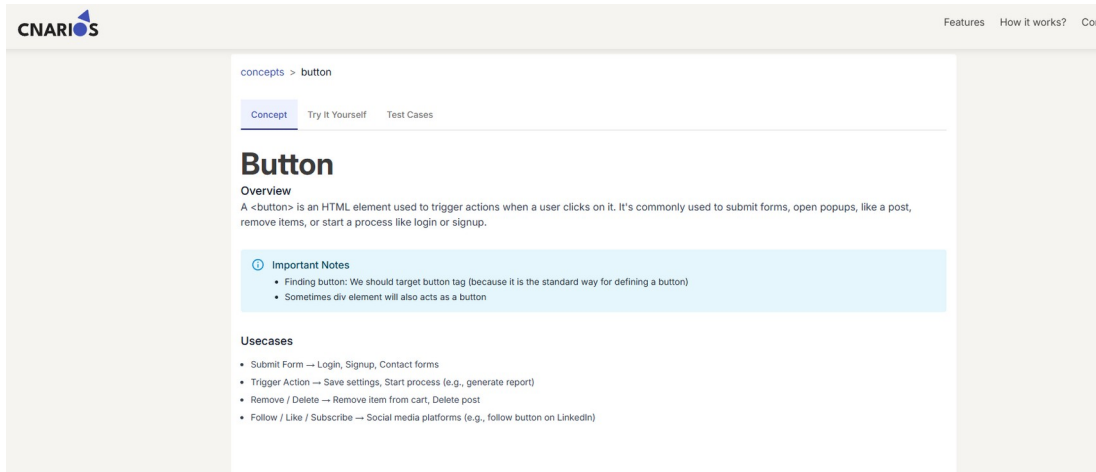
EV-MAN-06: formulario diligenciado con datos de prueba.

TCM-03: Apertura de nueva ventana

Se verificó la apertura de una nueva pestaña o ventana desde el concepto Multi Window.



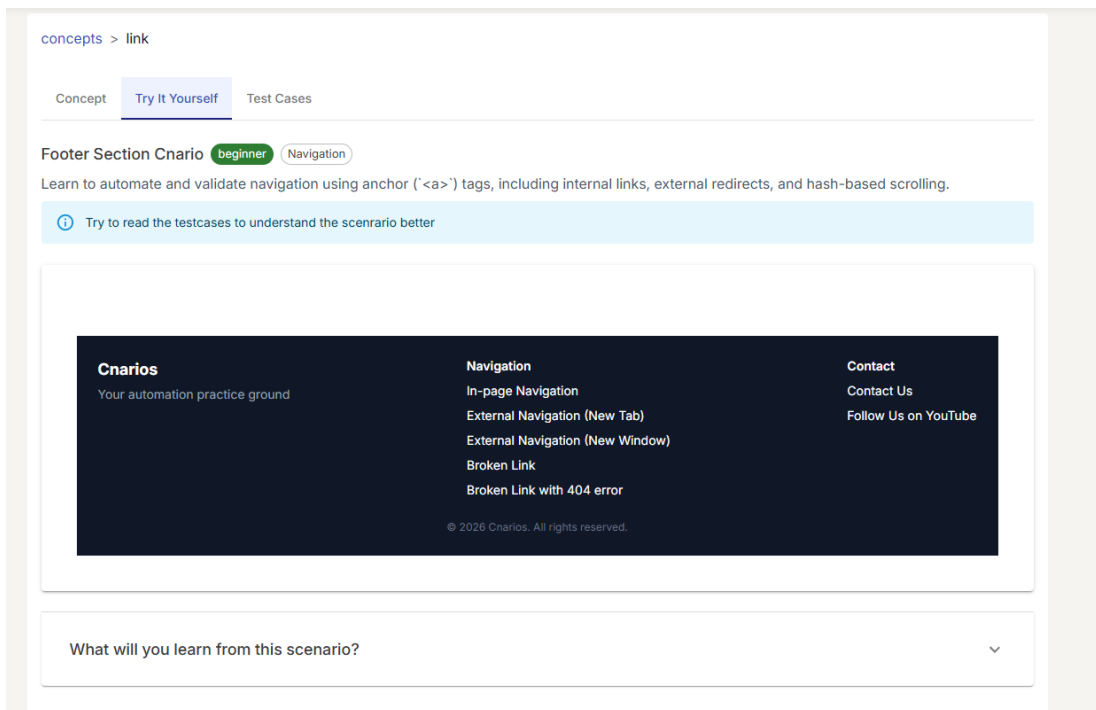
EV-MAN-07: sección Multi Window y enlaces disponibles.



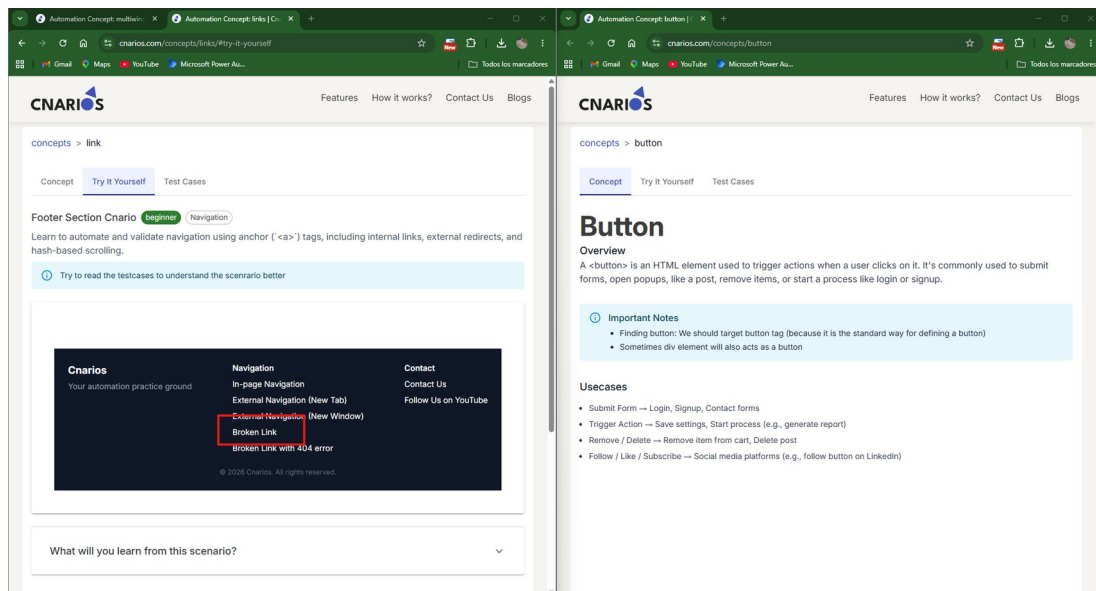
EV-MAN-08: nueva ventana/pestaña cargada correctamente.

TCM-04: Verificación de enlaces

Se revisaron enlaces funcionales y el escenario de broken link.



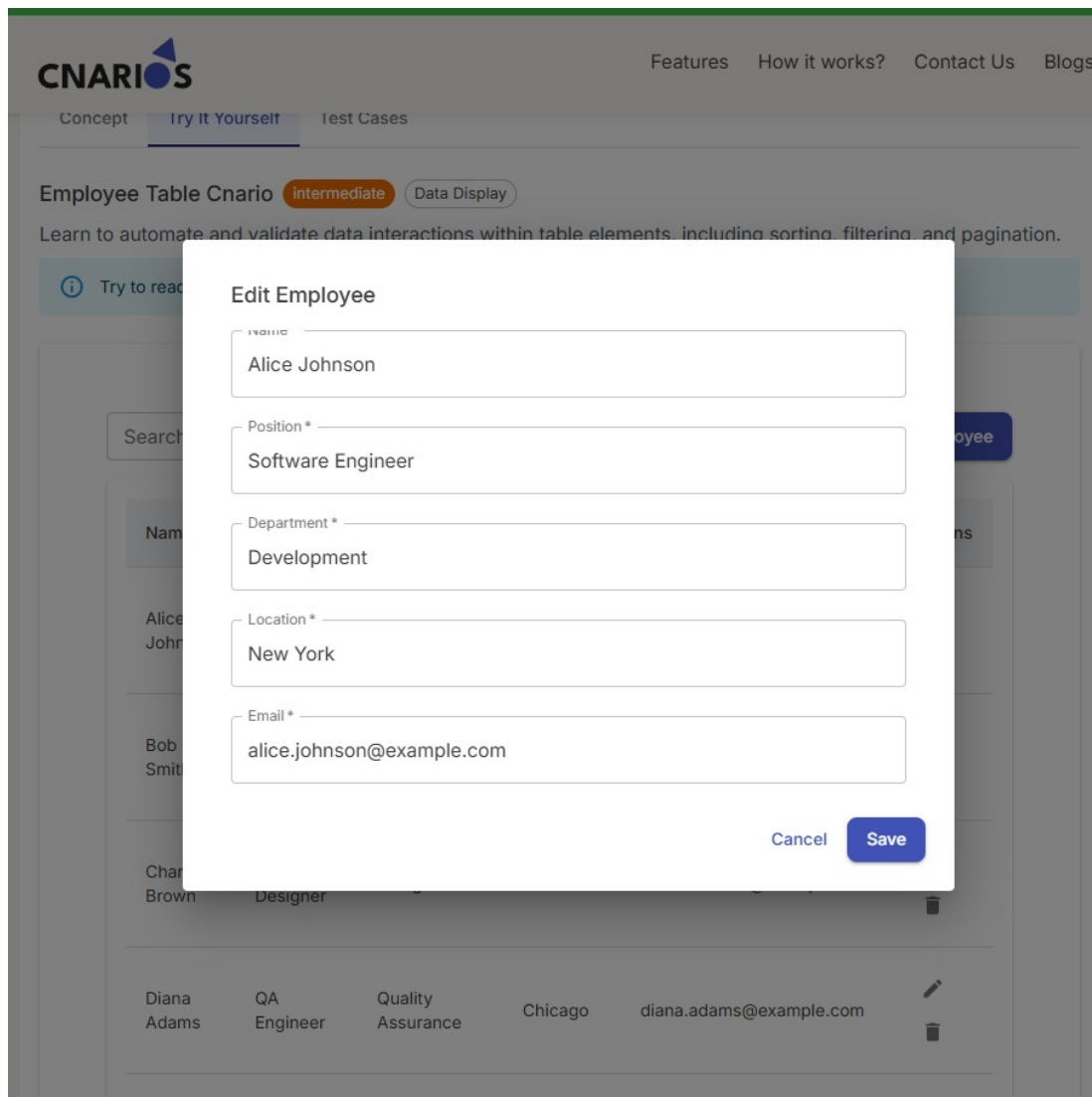
EV-MAN-09: escenario de enlaces.



EV-MAN-10: evidencia del enlace roto documentado.

TCM-05: Funcionalidad de tabla

Se validó tabla o componente interactivo disponible en el escenario.



EV-MAN-11: evidencia del componente interactivo validado.

TCM-06: Navegación del blog

Los artículos cargan correctamente y son accesibles desde la navegación principal.

[← Back](#)

HTML Basics for Automation

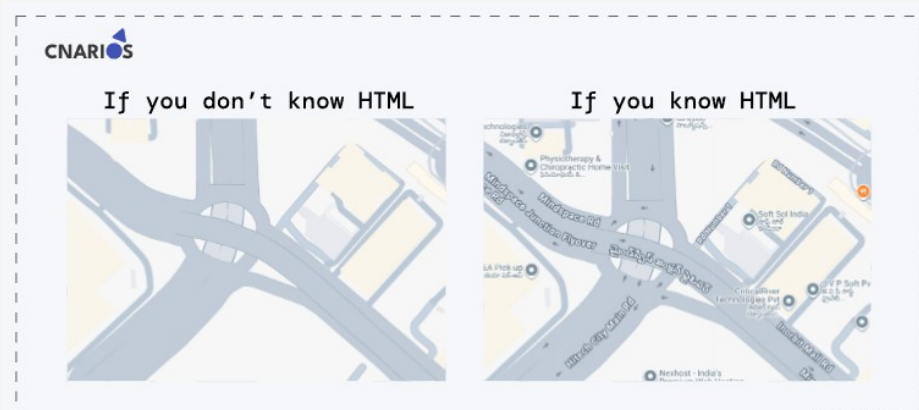
HTML Basics

Why HTML Matters in Web Automation?

We need to learn HTML for automation because it helps us understand the structure of a webpage, so we can correctly locate and interact with elements like buttons, inputs, and links.

What is HTML?

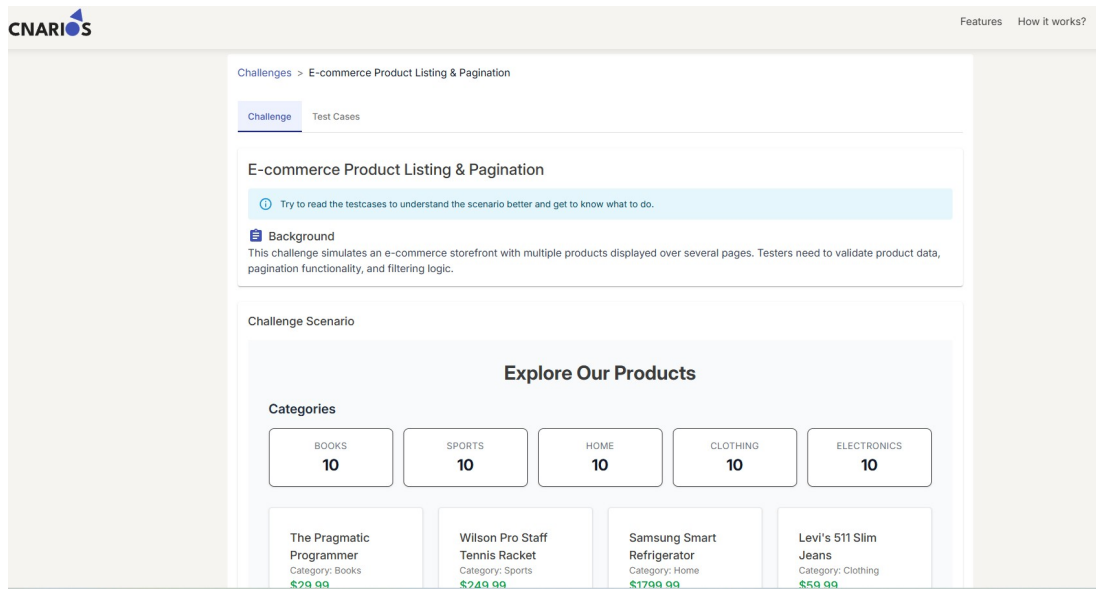
Using a city map without street names or landmarks is frustrating. You wouldn't know where you are or where to go next. HTML is like that map — it labels and organizes everything on a webpage. Without it, you're just guessing your way around.



EV-MAN-12: artículo del blog cargado correctamente.

TCM-07: Paginación

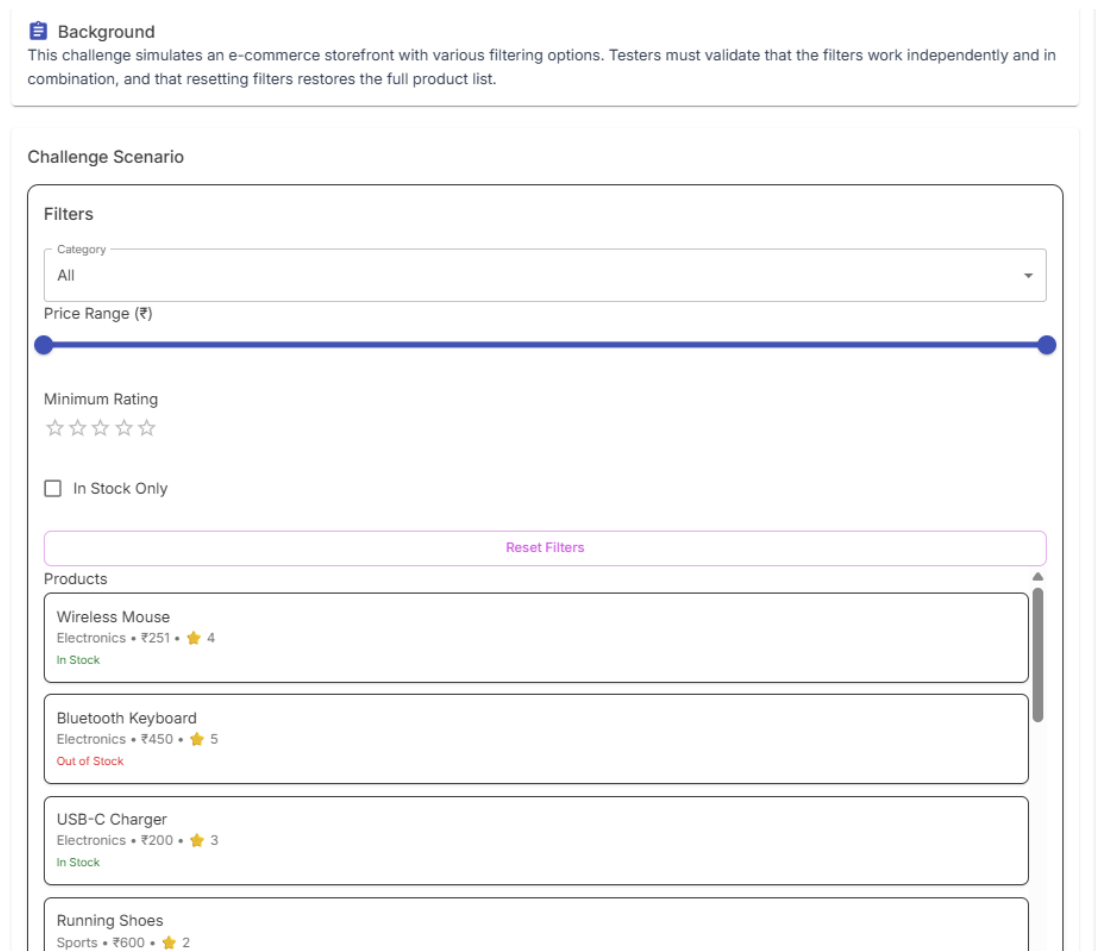
La lista de productos cambia al avanzar y vuelve al estado inicial al retroceder.



EV-MAN-13: desafío de paginación de productos.

TCM-08: Filtro por categoría

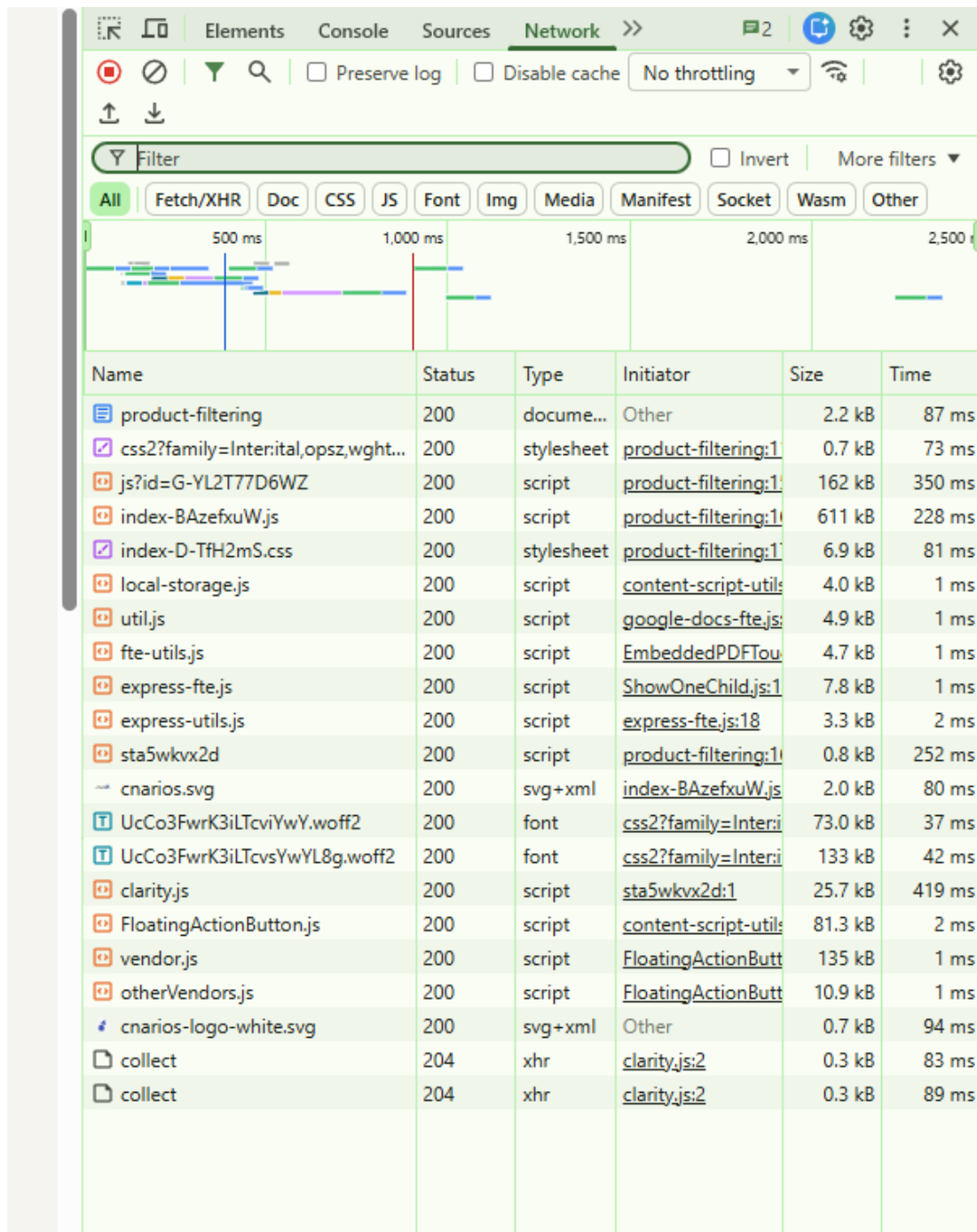
El filtro por categoría funciona y la opción de reinicio restaura la lista.



EV-MAN-14: filtro de productos aplicado.

TCM-09: Tiempos de carga

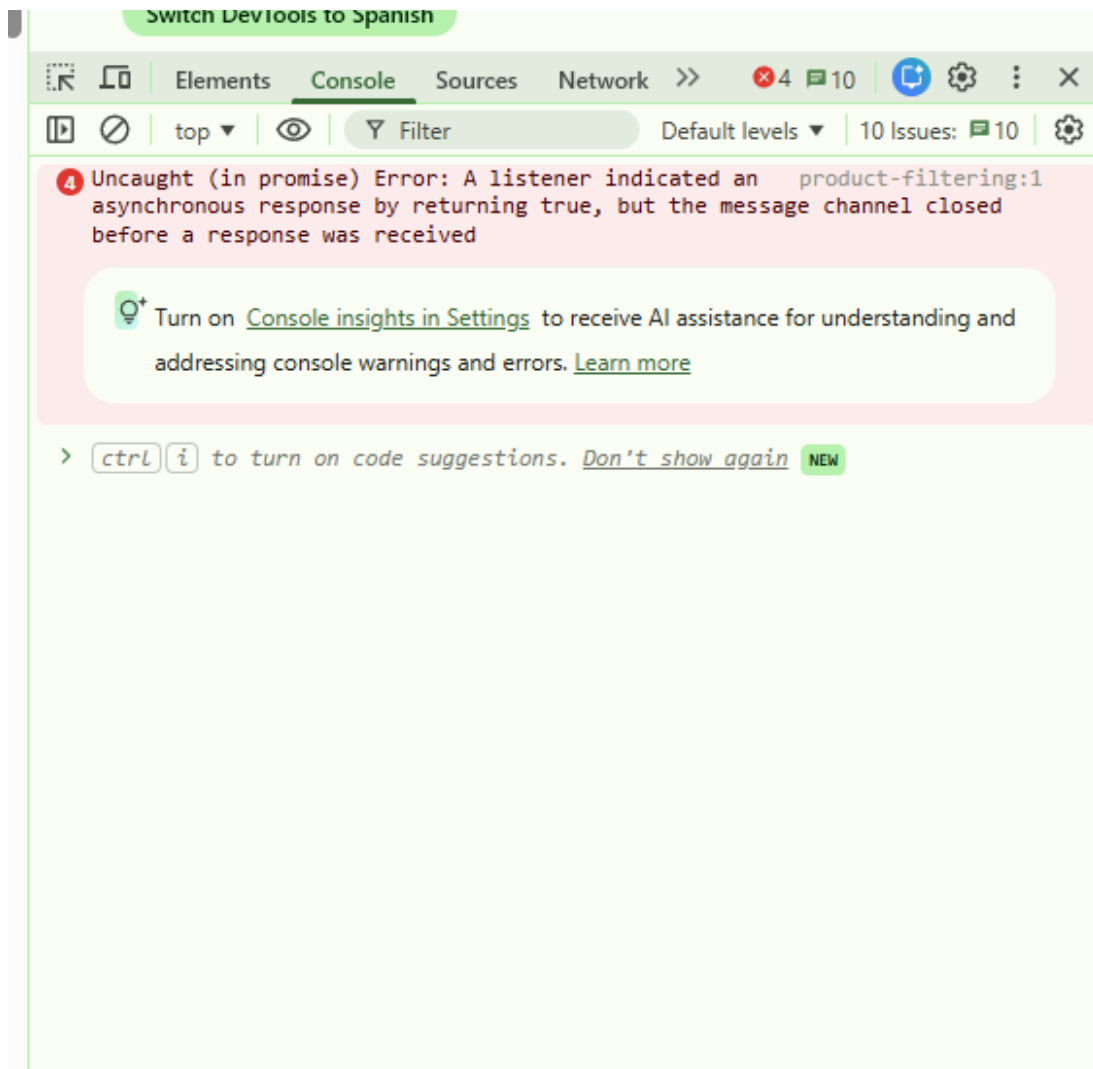
Las páginas principales cargan dentro del umbral definido de 3 segundos.



EV-MAN-15: medición de carga desde DevTools/Network.

TCM-10: Errores JavaScript / HTTP

No se observaron errores críticos de JavaScript ni respuestas HTTP 5xx durante la navegación principal.



EV-MAN-16: consola sin errores críticos durante navegación.

7. Automatización de pruebas

La automatización fue implementada con Playwright y TypeScript. Se utilizó el patrón Page Object Model para separar la lógica de navegación de los escenarios de prueba. Los scripts automatizados cubren cinco escenarios funcionales principales y pruebas no funcionales asociadas a tiempos de carga y errores graves de consola.

Elemento	Descripción
Herramienta	Playwright
Lenguaje	TypeScript
Patrón	Page Object Model
Navegadores	Chrome y Firefox
Comando	npm run test:compat
Reportes	evidencias/playwright-report/index.html y evidencias/playwright-results/results.json

7.1 Integración CI/CD

El repositorio incluye o propone un flujo de GitHub Actions para ejecutar instalación, pruebas automatizadas, revisión pasiva de seguridad y publicación de artefactos. En caso de habilitar SonarQube en CI/CD, se deben configurar los secretos SONAR_TOKEN y SONAR_HOST_URL.

```
npm ci
npx playwright install chrome firefox
npm run test:compat
npm run evidence:security
npm run report:build
```

8. Gestión de defectos

La gestión de defectos se documentó en formato Markdown dentro del repositorio, simulando un backlog verificable. Cada defecto incluye identificador, descripción, severidad, prioridad, estado, evidencia y recomendación.

ID	Descripción	Sev.	Prior.	Estado	Recomendación
DEF-01	Correo mailto posiblemente inválido	Baja	Media	Nuevo	Corregir dirección mailto y agregar prueba de enlace.
DEF-02	Iframe de pago sin confirmación visible	Media	Media	Analizado	Agregar mensajes de éxito/error accesibles.
DEF-03	Múltiples etiquetas canonical	Baja	Baja	Analizado	Renderizar una única canonical por ruta.
DEF-04	Cabeceras de seguridad incompletas	Media	Alta	Nuevo	Configurar CSP, frame-ancestors, nosniff, CORS y SRI.

8.1 Ciclo de vida representativo - DEF-04

Estado	Acción
Nuevo	Se detectan cabeceras faltantes durante revisión pasiva.
Analizado	Se clasifica como OWASP A05 Security Misconfiguration.
Reportado	Se recomienda CSP, frame-ancestors, nosniff y Referrer-Policy.
Resuelto pendiente	Desarrollo/infraestructura debe configurar cabeceras en servidor/CDN.
Cerrado pendiente	QA debe reejecutar ZAP y confirmar remediación.

9. Pruebas de seguridad

La evaluación de seguridad se realizó con base en OWASP Top 10. Por tratarse de un sitio público, se evitó ejecutar ataques activos agresivos y se priorizó un escaneo pasivo con OWASP ZAP Baseline. Además, se ejecutó SonarQube sobre el repositorio de automatización para medir calidad estática del código disponible.

9.1 OWASP ZAP Baseline

```
docker run --rm `
-v "${PWD}/evidencias/security:/zap/wrk:rw" `
zapproxy/zap-stable zap-baseline.py `
-t https://www.cnarios.com `
-r zap-baseline-report.html `
-J zap-baseline-report.json `
-m 2 -a -I
```

Resultado documentado: ZAP Baseline revisó 55 URLs, reportó 0 fallos nuevos y 16 advertencias nuevas. Las alertas principales se relacionan con Content Security Policy no configurada, CORS permisivo, cabecera anti-clickjacking ausente, SRI ausente y Permissions-Policy faltante.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

Total of 55 URLs
PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
PASS: In Page Banner Information Leak [10009]
PASS: Cookie No HttpOnly Flag [10010]
PASS: Cookie Without Secure Flag [10011]
PASS: Content-Type Header Missing [10019]
PASS: Information Disclosure - Debug Error Messages [10023]
PASS: Information Disclosure - Sensitive Information in URL [10024]
PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]
PASS: HTTP Parameter Override [10026]
PASS: Off-site Redirect [10028]
PASS: Cookie Poisoning [10029]
PASS: User Controllable Charset [10030]
PASS: User Controllable HTML Element Attribute (Potential XSS) [10031]
PASS: Viewstate [10032]
PASS: Directory Browsing [10033]
PASS: Heartbleed OpenSSL Vulnerability (Indicative) [10034]
PASS: Strict-Transport-Security Header [10035]
PASS: HTTP Server Response Header [10036]
PASS: Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037]
PASS: X-Backend-Server Header Information Leak [10039]
PASS: Secure Pages Include Mixed Content [10040]
PASS: HTTP to HTTPS Insecure Transition in Form Post [10041]
PASS: HTTPS to HTTP Insecure Transition in Form Post [10042]
PASS: User Controllable JavaScript Event (XSS) [10043]
PASS: Big Redirect Detected (Potential Sensitive Information Leak) [10044]

```

EV-SEC-01: ejecución de OWASP ZAP Baseline desde consola.

```

zap-baseline-report.json M X
evidencias > security > {} zap-baseline-report.json > ...
You, 1 hour ago [1 author (You)]
1  {
2    "@programName": "ZAP",
3    "@version": "2.17.0",
4    "@generated": "Fri, 15 May 2026 05:50:07",
5    "created": "2026-05-15T05:50:07.292388382Z",
6    "insights": [
7      {
8        "level": "Low",
9        "reason": "Warning",
10       "site": "",
11       "key": "insight.log.warn",
12       "description": "ZAP warnings logged - see the zap.log file for details",
13       "statistic": "1"
14     },
15     {
16       "level": "Info",
17       "reason": "Informational",
18       "site": "https://www.cnarios.com",
19       "key": "insight.code.2xx",
20       "description": "Percentage of responses with status code 2xx",
21       "statistic": "100"
22     },
23     {
24       "level": "Info",
25       "reason": "Informational",
26       "site": "https://www.cnarios.com",
27       "key": "insight.endpoint.ctype.application/javascript",
28       "description": "Percentage of endpoints with content type application/javascript",
29       "statistic": "2"
30     },
31     {
32       "level": "Info",
33       "reason": "Informational",
34       "site": "https://www.cnarios.com",
35       "key": "insight.endpoint.ctype.application/xml",
36       "description": "Percentage of endpoints with content type application/xml",
37       "statistic": "2"
38     },
39     {
40       "level": "Info",
41       "reason": "Informational",
42       "site": "https://www.cnarios.com",
43       "key": "insight.endpoint.ctype.image/svg+xml",
44       "description": "Percentage of endpoints with content type image/svg+xml",
45       "statistic": "2"
46     }
47   ]
48 }

```

EV-SEC-02: evidencia de reporte o configuración de alertas OWASP ZAP.

9.2 Vulnerabilidades, riesgos y mitigación

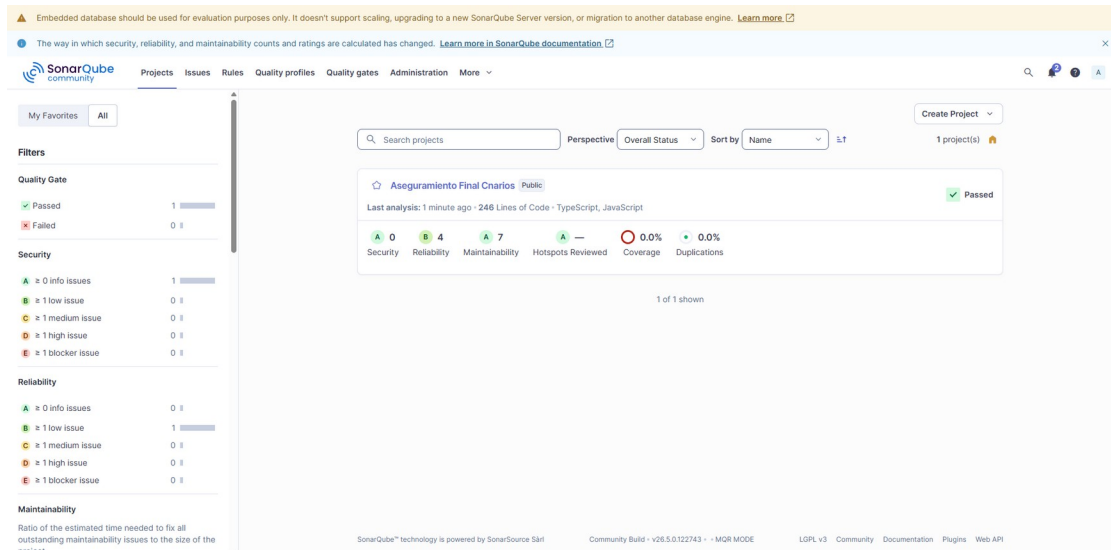
ID	Riesgo	Impacto	Mitigación
SEC-01	CSP ausente	Mayor exposición ante XSS si aparece una inyección.	Definir política restrictiva de Content-Security-Policy.
SEC-02	Anti-clickjacking ausente	El sitio podría ser embebido en	Usar frame-ancestors o X-Frame-

ID	Riesgo	Impacto	Mitigación
		frames no autorizados.	Options.
SEC-03	X-Content-Type-Options ausente	Riesgo de MIME sniffing.	Agregar X-Content-Type-Options: nosniff.
SEC-04	Referrer-Policy ausente	Posible filtración de URL de referencia.	Usar strict-origin-when-cross-origin.
SEC-05	Múltiples canonical	Ambigüedad SEO y mantenibilidad.	Emitir una canonical por ruta.
SEC-06	CORS permisivo	Riesgo mayor si se exponen datos sensibles.	Restringir orígenes permitidos.
SEC-07	SRI ausente	Riesgo si un proveedor externo es comprometido.	Usar Subresource Integrity e inventario de terceros.

9.3 SonarQube

Cnarios no publica su código fuente, por lo tanto SonarQube se ejecutó sobre el repositorio de automatización construido para esta entrega. El análisis permite medir mantenibilidad, bugs, vulnerabilidades, duplicación, cobertura y ratings del código de pruebas.

```
docker run --rm `
-e SONAR_HOST_URL=http://host.docker.internal:9000 `
-e SONAR_TOKEN=<TOKEN> `
-v "${PWD}:/usr/src" `
sonarsource/sonar-scanner-cli
```



EV-SONAR-01: dashboard o proyecto analizado en SonarQube.

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another d sec 1/1

SonarQube community Projects Issues Rules Quality profiles Quality gates Administration More

My account / Security

Security

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name Type Expires in

Enter Token Name Select Token Type 30 days Generate

New token "token-projecto-final-cnarios" has been created. Make sure you copy it now, you won't be able to see it again!

soa_3fe2a136d97f86e720d224f86e649ae8788c8

Name	Type	Project	Last use	Created	Expiration
codex-local-analysis-202605152205...	User		< 1 hour ago	May 15, 2026	-
codex-scan-20260515221227	User		< 1 hour ago	May 15, 2026	-
token-projecto-final-cnarios	User		Never	May 15, 2026	June 13, 2026

Enter a new password

EV-SONAR-02: métricas principales de SonarQube.

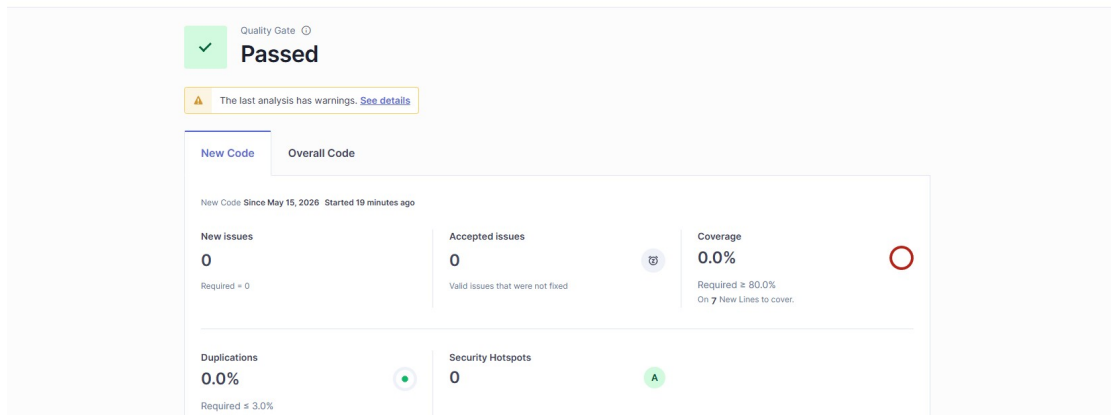
```
03:22:52.186 INFO No LCOV files were found using coverage/lcov.info
03:22:52.186 WARN No coverage information will be saved because all LCOV files cannot be found.
03:22:52.187 INFO Sensor JavaScript/TypeScript Coverage [javascript] (done) | time=873ms
03:22:52.187 INFO Sensor TextAndSecretsSensor [text]
03:22:52.113 INFO Available processors: 20
03:22:52.113 INFO Using 20 threads for analysis.
03:22:52.275 INFO Start fetching files for the text and secrets analysis
03:22:52.279 INFO Using Git CLI to retrieve dirty files
03:22:52.285 WARN Retrieving only language associated files, make sure to run the analysis inside a git repository to make use of inclusions specified via "sonar.text.inclusion
03:22:52.286 INFO Starting the text and secrets analysis
03:22:52.287 INFO 7 source files to be analyzed for the text and secrets analysis
03:22:52.300 INFO 7/7 source files have been analyzed for the text and secrets analysis
03:22:52.301 INFO Sensor TextAndSecretsSensor [text] (done) | time=194ms
03:22:52.304 INFO ----- Run sensors on project
03:22:52.372 INFO Sensor Zero Coverage Sensor
03:22:52.376 INFO Sensor Zero Coverage Sensor (done) | time=3ms
03:22:52.376 INFO ----- Gather SCA dependencies on project
03:22:52.378 INFO Dependency analysis skipped
03:22:52.382 INFO SCM Publisher SCM provider for this project is: git
03:22:52.382 INFO SCM Publisher 3 source files to be analyzed
03:22:52.424 WARN Thread[#78,ForkJoinPool.commonPool-worker-1,5,main]: got smaller file timestamp on /usr/src (C:\), /usr/src/.git: 2026-05-16T03:22:52Z < 2026-05-16T03:22:52.4
orting measurement
03:22:52.811 INFO SCM Publisher 0/3 source files have been analyzed (done) | time=428ms
03:22:52.811 WARN Missing blame information for the following files:
03:22:52.811 WARN * scripts/report/build-report.ts
03:22:52.811 WARN * src/utils/evidence.ts
03:22:52.811 WARN * scripts/performance/cnarios-smoke.js
03:22:52.812 WARN This may lead to missing/broken features in SonarQube
03:22:52.814 INFO CPD Executor Calculating CPD for 5 files
03:22:52.818 INFO CPD Executor CPD calculation finished (done) | time=4ms
03:22:52.835 INFO SCM revision ID 'a7453940022che880c3f4fbc74a64848c8a78e44'
03:22:52.896 INFO Analysis report generated in 74ms, dir size=315.3 kB
03:22:52.913 INFO Analysis report compressed in 16ms, zip size=60.9 kB
03:22:52.939 INFO Analysis report uploaded in 25ms
03:22:52.940 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://host.docker.internal:9001/dashboard?id=aseguramiento-final-cnarios
03:22:52.940 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
03:22:52.940 INFO More about the report processing at http://host.docker.internal:9001/ap1/ce/task?id=4e0bc506-fdfe-4584-bf5c-17c4908ade8b
03:22:53.022 INFO Analysis total time: 23.851 s
03:22:53.023 INFO SonarScanner Engine completed successfully
03:22:53.351 INFO EXECUTION SUCCESS
03:22:53.352 INFO Total time: 34.776s
PS C:\Users\jivan\Desktop\ASEGURAMIENTO_FINAL>
```

EV-SONAR-03: ejecución del sonar-scanner desde consola.

Overview

250 Lines of Code · Version 1.0 · Last analysis 2 minutes ago

Set as homepage



EV-SONAR-04: detalle de medidas o calidad del proyecto en SonarQube.

10. Métricas de calidad

Las métricas permiten interpretar el estado de calidad del sistema y del proceso de pruebas. Se midió cobertura de requisitos, cantidad de casos diseñados, casos automatizados, tasa de aprobación, defectos registrados, tiempo de ejecución, rendimiento p95 y resultados de SonarQube.

Métrica	Fórmula/Fuente	Resultado
Cobertura de requisitos	Requisitos cubiertos / requisitos totales	10/10 = 100 %
Casos manuales diseñados	Total documentado	10
Casos automatizados	Total ejecutable	5 funcionales + 2 no funcionales
Tasa de aprobación	Casos aprobados / ejecutados	14/14 = 100 %
Defectos registrados	Backlog documentado	4
Densidad de defectos	Defectos funcionales / módulos	2 / 4 módulos
Tiempo de ejecución	Duración total suite	19.95 s
Rendimiento p95	Percentil 95 HTTP	95.62 ms
Fallos HTTP k6	Fallos / requests	0 %

10.1 Métricas SonarQube

Indicador	Resultado
Bugs	0
Vulnerabilidades	0
Security Hotspots	0
Code Smells	7
Duplicación	0.0 %
Coverage	0.0 %
Reliability Rating	A
Maintainability Rating	A
Security Rating	A

La cobertura aparece en 0.0 % porque el análisis estático se aplica sobre el repositorio de pruebas y no sobre el producto Cnarios, el cual se evalúa como caja negra. Como mejora futura se recomienda agregar pruebas unitarias sobre utilidades internas del framework de automatización para elevar esta métrica.

11. Reflexión y lecciones aprendidas

El proyecto permitió comprender que el aseguramiento de calidad no se limita a verificar que una página cargue. Fue necesario planificar, diseñar casos, automatizar escenarios, ejecutar pruebas manuales, medir rendimiento, evaluar seguridad, registrar defectos y analizar métricas.

Uno de los principales retos fue automatizar un sitio externo sin tener control sobre su código fuente. Para reducir la fragilidad se utilizaron selectores basados en roles, textos visibles y una estructura Page Object Model. Otro reto fue la seguridad, debido a que el análisis debía ser responsable y no invasivo.

Como aprendizaje principal se destaca la importancia de respaldar cada afirmación con evidencia. Las capturas, logs, reportes HTML, JSON y dashboards permiten demostrar que las pruebas fueron ejecutadas realmente. Para futuros proyectos se recomienda ampliar cobertura, incluir pruebas móviles, mejorar métricas de cobertura y ejecutar escaneos de seguridad más profundos con autorización del propietario.

12. Conclusiones

El proceso de aseguramiento de calidad aplicado sobre Cnarios permitió validar funcionalidades principales mediante pruebas manuales y automatizadas, evaluar atributos no funcionales como rendimiento, estabilidad y seguridad, y registrar defectos con evidencia verificable.

Los resultados obtenidos muestran una ejecución satisfactoria de los casos funcionales y automatizados, tiempos de respuesta dentro del umbral definido, y hallazgos de seguridad relacionados principalmente con configuración defensiva. La entrega demuestra que el aseguramiento de calidad requiere planificación, trazabilidad, automatización, análisis técnico y documentación clara, no solo ejecutar herramientas sin interpretar sus resultados.

Como recomendación general, se propone mejorar las cabeceras de seguridad, corregir enlaces de correo, agregar mensajes visibles en formularios interactivos, mantener una canonical única por ruta y extender la automatización a pruebas móviles y pruebas de accesibilidad.

13. Anexos

Anexo	Elemento	Ruta o enlace
A	Repositorio GitHub	https://github.com/ziuvar/PROYECTO-FINAL-ASEGURAMIENTO.git
B	Casos de prueba	docs/casos_prueba.md
C	Defectos	docs/defectos.md
D	Métricas	docs/metricas.md
E	Resumen de ejecución	docs/resumen_ejecucion.md
F	Reporte Playwright	evidencias/playwright-report/index.html
G	Resultados Playwright	evidencias/playwright-results/results.json
H	Reporte k6	evidencias/performance/k6-summary.json
I	Reporte OWASP ZAP	evidencias/security/zap-baseline-report.html
J	Reporte ZAP JSON	evidencias/security/zap-baseline-report.json
K	Dashboard SonarQube	evidencias/sonarqube/sonarqube-dashboard.png
L	Configuración Playwright	playwright.config.ts
M	Configuración SonarQube	sonar-project.properties

Nota: Las evidencias visuales fueron insertadas dentro del informe y también se conservan como recursos de apoyo en la carpeta evidencias del repositorio.